

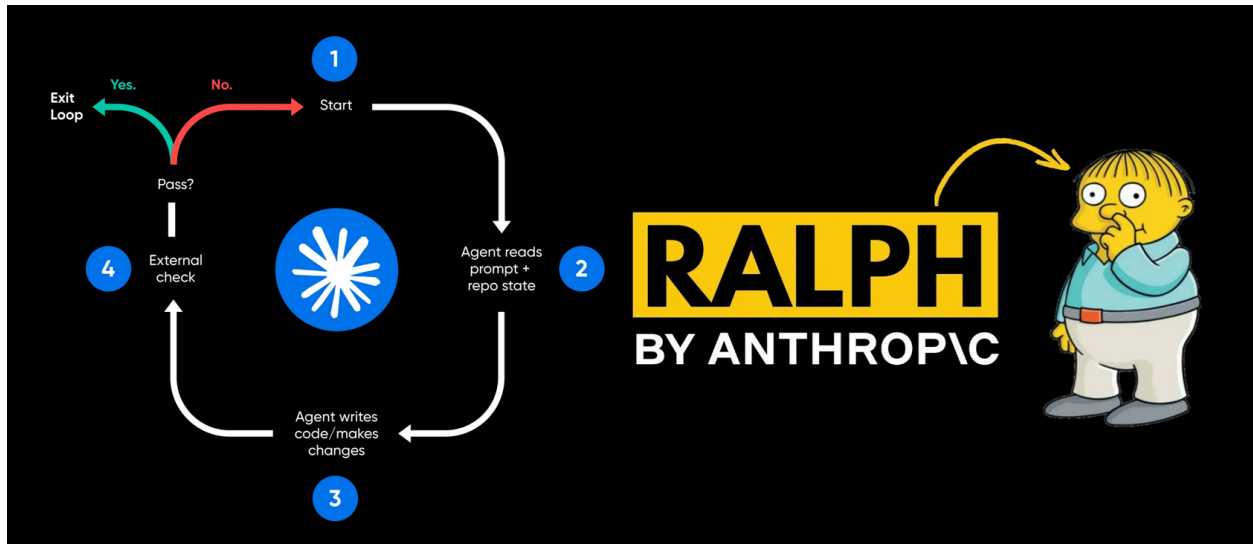
Week 10; Host-Native Harness Engineering

1. What is Harness Engineering

- AI 에이전트가 복잡한 작업을 안전하고 효율적으로 완수할 수 있도록, 프롬프트나 모델을 넘어서 외부 모실행 환경(도구, 데이터, 검증 파이프라인, 안전장치)을 구조적으로 설계하고 통제하는 기술.



하네스 엔지니어링의 진화



랄프 루프의 단계

- Phase 1 : Jobs To Be Done → AI Agent와 사람이 대화하며 /spec 폴더에 명세서를 만든다
- Phase 2 : AI 에이전트가 기존 코드 구현체와 명세서를 비교한 뒤 Plan을 세운다

while:

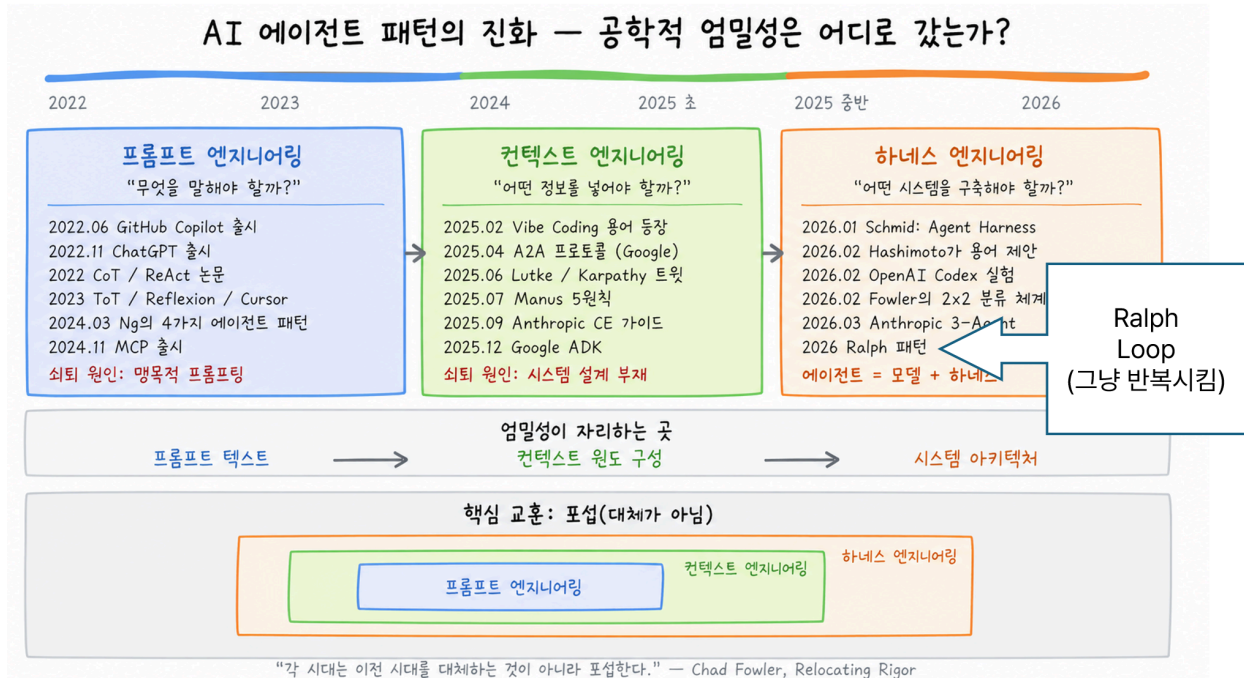
begin:

- Phase 3 : AI 에이전트가 1개의 Plan 체크리스트에 대해 작업 → 검증 → 계획 업데이트 → 커밋 → 종료 절차를 밟는다. 한 작업이 끝나면 새 에이전트가 다음 체크리스트의 작업을 진행한다.

:end

생각 해 볼것:

- 명시적인 달성 조건이 있어야함
- 최대 작업 상한이 있어야 함



2. 기본 구성 요소

- 하네스 엔지니어링은 에이전트가 작업을 안정적으로 수행하도록 만드는 실행 구조이다.
- 단순히 프롬프트를 잘 작성하는 것이 아니라, 작업의 목표·절차·기록·선호를 분리해 관리한다.

책임	질문	파일 예시
Contract	무엇이 끝났는가?	<code>TASK.md</code>
Procedure	어떻게 진행하는가?	Skill
Journal	지금까지 무엇을 했는가?	<code>Log</code> : 또는 MCP
Preference	어떤 방식으로 일하는가?	<code>PROFILE.md</code> , <code>AGENTS.md</code> , <code>CLAUDE.md</code>

Contract가 없는 Procedure는 무한 루프이고, Procedure가 없는 Contract는 죽은 문서다.

즉, 절차만 있으면 에이전트가 언제 멈춰야 할지 모른다.

반대로 계약만 있으면 에이전트가 어떻게 진행해야 할지 모른다.

AGENTS.md

- > 매 세션 시작 시 이 파일과 `journal.md`를 먼저 읽어라.
- > 이 파일이 너의 작업 방식 전부다.

0. 파일 구조

- `specs/` - 사용자와 합의한 요구사항 명세 (= 무엇을 만들 것인가)
- `TASK.md` - 지금 해야 할 작업 체크리스트
- `journal.md` - 지금까지 한 일의 기록 (append only, 절대 수정 금지)
- `AGENTS.md` - 이 파일

1. 작업 시작 전 (계획 단계)

- 프로젝트가 git으로 관리되고 있지 않으면 `git init`을 먼저 실행하라.
- `specs/` 폴더에서 사용자와 함께 계획을 세워라.
- 소크라테스적 질문으로 요구사항을 명확히 하라. ****가정하지 말고 물어라.****
- 사용자가 "이정도면 충분하다"라고 ****명시적으로**** 마감을 선언하기 전까지, `TASK.md`의 체크리스트를 계속 다듬어라.
- 마감 선언 전에는 단 한 줄의 production 코드도 쓰지 마라.

2. 매 작업 (구현 단계)

매 반복은 다음 순서를 정확히 따라라:

1. `journal.md`의 마지막 5줄을 읽어 직전 상태를 확인하라.
2. `TASK.md`에서 가장 위의 미완료 항목 ****단 한 개****만 골라라.
3. 관련 코드를 먼저 읽어라. 추측보다 탐색이 우선이다.
4. 구현하라.
5. 검증을 실행하라: `npm test && npm run lint && npm run build`
(실패하면 통과할 때까지 수정. 테스트를 건너뛰지 마라.)
6. git commit. 메시지에 ****"무엇을"**이 아니라 **"왜"**를 적어라.
7. `TASK.md`의 해당 항목을 체크하라.
8. `journal.md`에 한 줄 추가하라:
`- [YYYY-MM-DD HH:MM] <한 일 한 줄 요약> - <결과>`
9. 종료하라. 다음 작업은 다음 반복에서.

3. 절대 하지 마라

- ❌ 한 반복에서 두 개 이상의 작업을 처리하기
- ❌ 실패한 테스트를 주석 처리하거나 skip 처리하기
- ❌ `specs/` 없이 코드를 작성하기
- ❌ `journal.md`를 수정하거나 삭제하기 (오직 append)
- ❌ 사용자의 명시적 승인 없이 새 의존성 추가하기
- ❌ 같은 접근을 3번 실패한 뒤에도 시도 계속하기 → 멈추고 보고하라

4. 막히면

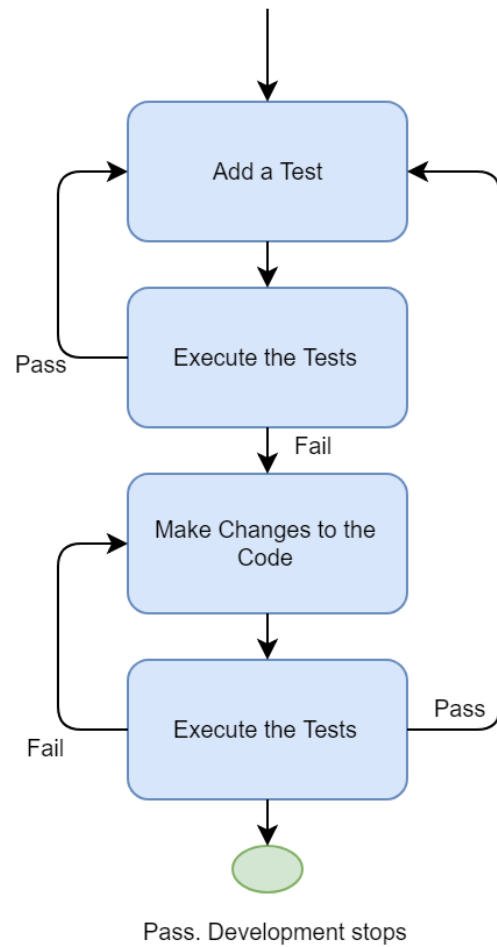
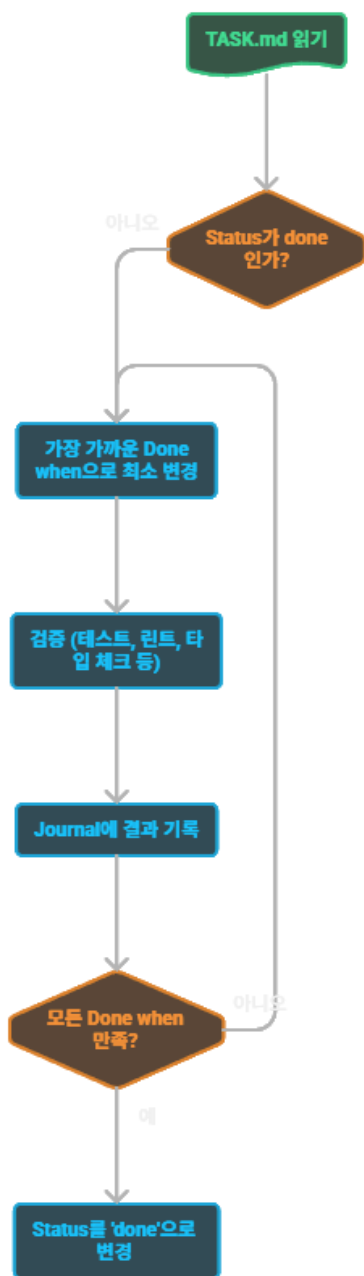
- 명세가 모호하면 추측하지 말고 사용자에게 질문하라.
- 같은 시도가 3번 실패하면 멈추고 `journal.md`에 상황을 적은 뒤 사용자에게 보고하라.

3. Contract-Driven Iteration

Contract-Driven Iteration은 에이전트가 작업을 반복해서 끝내게 만드는 방식이다.

에이전트가 Contract를 읽고, Done when을 만족할 때까지 반복한다.

1. `TASK.md` 를 읽는다.
2. `Status` 가 `done` 이면 멈춘다.
3. 아직 `open` 이면 가장 가까운 Done when을 향해 최소 변경을 한다.
4. 테스트, 린트, 타입 체크 등으로 검증한다.
5. 결과를 Journal에 한 줄 기록한다.
6. 모든 Done when이 만족되면 `Status` 를 `done` 으로 바꾼다.
7. 아니면 다시 반복한다.



TDD

4. How to use

4.1 Contract 작성법

보통 `TASK.md` 에 아래와 같이 작성

Status: open

Goal: Add timeout parameter to `fetch_user()` with 5s default.

Done when:

- ``pytest tests/test_fetch_user.py -q`` passes
- New parameter has docstring entry
- Existing callers still work

Log:

-

좋은 Contract 조건

조건	설명
Goal은 한 문장	두 문장이면 작업이 두 개인 경우가 많다
Done when은 측정 가능	<code>works well</code> 은 금지
검증 명령은 구체적	<code>pytest</code> 보다 <code>pytest tests/test_foo.py -q</code>
Done when은 1~4개	많으면 작업을 쪼갬다
부정 가드 포함	새 의존성 금지, 기존 기능 유지 등

4.2 Skill이란?

Skill은 에이전트가 특정 작업을 어떻게 수행할지 정의한 마크다운 파일

AI Agent는 작업 맥락에 맞는 Skill을 자동으로 찾아 적용할 수 있음

구조: SKILL.md 파일 + 필요 시 보조 파일들이 한 폴더에 묶인 형태.

/skills

- 확 하고 아키텍처가 떠오르지 않네요, 핵심 컴포넌트들은 무엇이 있을까요 ?
- 아하, 대략적인 다이어그램을 작성해주실 수 있을까요 ?
- Js를 사용하지 말고, 간단히 mermaid로 작성 후 png 파일로 show 해주세요.

필요할 때 마다
매번 이 행위가 반복됨

/skill-creator 제가 요청한 무엇이든지, "/diagram"으로 호출하면 (의도 분해 -> 추상화된 mermaid 생성 -> 저에게 PNG 이미지 형태로 출력) 방식으로 하는 Skill을 만들어주세요.

+

Opus 4.7 적용형



✳ 의도 분해와 Mermaid 렌더링 방식 검토 중

Read the skill-creator SKILL.md to understand how to create skills

View the truncated middle section

diagram

/diagram

+

올래서 커멘트 /diagram 으로 시작하는 모든 요청을 처리합니다. 사용자의 지면의 명령들 (1) 의도 분해 (2) 추상화된 Mermaid 다이어그램 생성 (3) PNG 이미지로 렌더링하는 순서로 처리합니다. 시스템에 커멘트, 시스템 (클로우, 상태 머신, ER, 클래스, 사용자 역할 등 모든 종류의 시각화 요청에 사용됩니다. 사용자가 /diagram 이라는 올래서 커멘트로 시작하면 두 조건이 충족될 때까지 기다립니다. 다른 이미지 생성 도구 (dialpad 등)를 추가하고, 시스템의 렌더 스크립트만 사용해야 합니다 — 그제야 관련된 주석과 품질과 한국에 폰트 처리가 보장됩니다.



4.3 Journal 작성법

- Journal은 반복마다 무엇을 했는지 남기는 기록으로, 누적되고, 수정하지 않는 것이 좋음
- Codex와 Claude에는 현재 MEMORY 기능이 존재하며, 사실 이것이 이 Journal의 기능을 대신하게 됨.

Journal에 들어갈 것

항목	예시
무엇을 했는가	Added timeout=5 default
왜 했는가	Needed to satisfy Done when
결과는 무엇인가	pytest passes / failed / reverted

어디에 둘까?

방식	위치	적합한 경우
In-file	TASK.md 의 Log:	단일 세션 작업
External	MCP append/read	세션을 넘어 이어지는 작업

5. 가장 흔한 실패와 해결

실패	문제	해결
Done when이 모호함	모델이 자기 판단으로 완료 처리	명령어로 검증 가능하게 작성
자기 결과를 자기가 평가	항상 후하게 평가	테스트, 린트, 별도 evaluator 사용
Contract가 너무 큼	중간에 길을 잃음	작은 Contract 여러 개로 분리
Plan 없이 Code	영뚱한 파일 수정	Explore와 Plan 먼저 수행
MCP에 execute 도구 추가	외부 wrapper가 됨	MCP는 append/read만 사용
단일 작업을 멀티로 처리	비용 증가, 디버깅 어려움	단일로 풀리면 단일로 처리
Done when을 수정함	기준을 낮춰 완료한 척함	Done when 수정 금지

6. OpenAI Goal

사용자가 /goal 명령어를 통해 목표 설정 뒤, 끝날때까지 계속하여 작업을 이어가는 기능



현재 Experimental 단계보다도 앞선, UnderDevelopment 단계로 Public Release가 아님.

아래와 같이 .codex 폴더의 config.toml 에서 기능을 on 할 수 있음.

```
[features]
goals = true
```

이후 codex를 실행하여 /goal 명령을 입력하면 정상적으로 작동하는것을 확인할 수 있음.

```
>_ OpenAI Codex (v0.128.0)

model:      gpt-5.5 high  fast  /model to change
directory: ~

Tip: Try the Codex App. Run 'codex app' or visit https://chatgpt.com/codex?app-landing-page=true

⚠Under-development features enabled: goals. Under-development features are incomplete and may behave unpredictably. To
  suppress this warning, set 'suppress_unstable_features_warning = true' in C:\Users\user\.codex\config.toml.

> /goal|

/goal set or view the goal for a long-running task
```

목표 달성 시 소요시간 등 정보 확인 가능

```
• Goal active Objective: 안녕
• 안녕하세요. 무엇을 도와드릴까요?

  목표 완료 처리했습니다. 소요 시간: 5초.

> |improve documentation in @filename
gpt-5.5 high fast · ~                               Goal achieved (5s)
```

Command	Action
/goal <목표>	새 목표 생성 및 루프 시작
/goal pause	현재 목표를 paused 상태로 전환
/goal resume	paused 상태를 다시 재
/goal clear	현재 목표 제거

에이전틱 코딩 패턴

기본 패턴 5가지

패턴	의미	예시
Prompt Chaining	앞 단계 결과를 다음 단계 입력으로 사용	스펙 작성 → 테스트 작성 → 구현
Routing	입력 종류에 따라 다른 경로 선택	문서 수정은 가벼운 모델, 보안 이슈는 강한 모델
Parallelization	독립 작업을 동시에 실행	여러 파일을 나누어 수정
Orchestrator-Workers	관리자가 작업을 쪼개 워커에게 배분	영향받는 파일을 찾고 각 워커에게 할당
Evaluator-Optimizer	만들고 평가하고 다시 고침	테스트 통과할 때까지 구현 반복

코딩에서 자주 쓰는 패턴

이름	흐름	핵심
EPCC	Explore → Plan → Code → Commit	바로 코딩하지 않고 먼저 읽고 계획한다
TDD	Red → Green → Refactor	실패 테스트를 먼저 만들고 통과시킨다
Visual Iteration	생성 → 화면 확인 → 수정	UI는 실제 화면으로 검증한다
Three-Agent Harness	Planner → Generator → Evaluator	계획, 구현, 평가를 분리한다

어떤 패턴을 선택할까?

상황	선택
단계 간 의존성이 있다	Prompt Chaining
독립 작업이 여러 개다	Parallelization
작업 수를 미리 모른다	Orchestrator-Workers
결과를 자동 검증할 수 있다	Evaluator-Optimizer
입력 유형별 처리가 다르다	Routing
단일 에이전트로 충분하다	단일 에이전트